

The EMCal π^0/η Smearing / Resolution-Matching Framework

An Explainer of the MC-to-Data Resolution Code Chain
Run-24 $p+p$ Non-Linearity Study (sPHENIX)

Code walkthrough — B. Seidlitz collaborator code, read-only

2026-06-17

Abstract

This document explains the *system* of code that “smears” (artificially degrades) the energy and position resolution of simulated EMCal clusters so that the reconstructed π^0 and η peaks in Monte Carlo (MC) match those measured in Run-24 $p+p$ data. The core problem: **Pythia/GEANT4 MC has a better (narrower) calorimeter resolution than the real detector**, so a direct data/MC comparison of meson mass and width is biased. The fix is to widen the MC by an amount equal to the *quadrature difference* between the data and MC resolution curves. This explainer focuses on the smearing/resolution path through the production engine `pi0EtaByEta.cc`, situates it in the three-stage production chain, and gives the governing equations. It is meant to be read for *flow and intent*, not line-by-line.

Contents

1	The one-paragraph mental model	1
2	The physics: why smear, and by how much	1
2.1	The resolution function	1
2.2	The extra smearing applied to MC	2
2.3	Representative parameter sets	2
3	Framework layout: three stages joined by a file	2
4	The engine in detail: <code>pi0EtaByEta.cc</code>	3
4.1	Loading the smear curves (<code>Init</code> , around line 206)	3
4.2	Booking the smeared histograms (<code>Init</code> , line 159)	3
4.3	Applying the smear in the event loop (line 738)	4
4.4	The smearing kernel: <code>smearPhot()</code> (line 863)	5
4.5	Decoding the six smear indices	5
5	Where <code>CaloCalibEmcPi0</code> fits in	6
6	Standalone toy cross-checks (not the production result)	6
7	How to run the chain (summary)	7

One-line dependency map**7****Key takeaways****7**

1 The one-paragraph mental model

Real calorimeter clusters are blurry; MC clusters are too sharp. To compare them fairly you must blur the MC by the right amount. “The right amount” is computed once, offline, as a function of photon energy E and stored as ROOT TF1 curves in a parameter file. At reconstruction time, every MC photon is passed through a function `smearPhot()` that (i) reads the stored width at that photon’s E , (ii) multiplies the photon energy by a Gaussian random number of that width, and (iii) nudges the photon (η, ϕ) direction by a second Gaussian. The smeared photons are recombined into a π^0/η candidate, and the resulting (now-realistically-broadened) mass is histogrammed. A downstream fitting macro then reads those histograms and reports mass and width versus p_T , plus a systematic band from running several smearing variations.

2 The physics: why smear, and by how much

2.1 The resolution function

Calorimeter energy resolution follows the standard three-term form (with E in GeV):

$$\frac{\sigma(E)}{E} = \sqrt{\frac{p_0^2}{E} + \frac{p_1^2}{E^2} + p_2^2} \equiv \frac{p_0}{\sqrt{E}} \oplus \frac{p_1}{E} \oplus p_2, \quad (1)$$

where $\oplus$ denotes addition in quadrature and

- p_0 = stochastic / sampling term ($a/\sqrt{E}$),
- p_1 = noise term (b/E),
- p_2 = constant term (c).

2.2 The extra smearing applied to MC

MC is already at its (good) MC resolution. To push it up to the (worse) data resolution, you add in quadrature only the *difference*:

$$\sigma_{\text{extra}}(E) = \sqrt{\max(0, \sigma_{\text{data}}^2(E) - \sigma_{\text{MC}}^2(E))} \quad (2)$$

and the smearing is a Gaussian draw

$$\text{energy smear} \sim \mathcal{N}(1, \sigma_{\text{extra}}(E)) \quad (\text{multiplicative on } E). \quad (3)$$

Because $\sigma_{\text{data}}^2 - \sigma_{\text{MC}}^2$ is taken under a $\max(0, \cdot)$, the smear vanishes wherever MC already matches or exceeds data; in the nominal set it is ≈ 0 below ~ 17 GeV and rises to a few percent at high E_T . This is exactly the curve precomputed and stored as a TF1.

Source	p_0 (stoch.)	p_1 (noise)	p_2 (const.)
Pythia intrinsic / MC	0.185	0.00	0.040
Nominal data target	0.15	0.05	0.05
“wider-data” variation	0.13	0.08	0.08
Tuned “smear11” set	0.180	0.020	0.015 (+ $E_{\text{scale}} \times 0.998$)
Test-beam single γ	0.16	–	0.05

Table 1: Resolution-term values gathered from the calibration talks and the PPG12 analysis note (Sec. 5.2). The MC is intrinsically narrow ($p_0 \approx 0.185$, tiny constant 0.040); the “8%” smearing referenced in the talks is the extra ~ 0.08 term needed to widen MC to data. Position smearing is parameterised similarly: $\sigma_\psi(E) = 0.0030/\sqrt{E} \oplus 0.0040$ (tuned).

2.3 Representative parameter sets

3 Framework layout: three stages joined by a file

The production smearing is *not one program*. It is three stages that hand a single ROOT file (the TF1 parameter file) and a set of histograms down a chain. **The bridge between stages is a file, not a function call.**

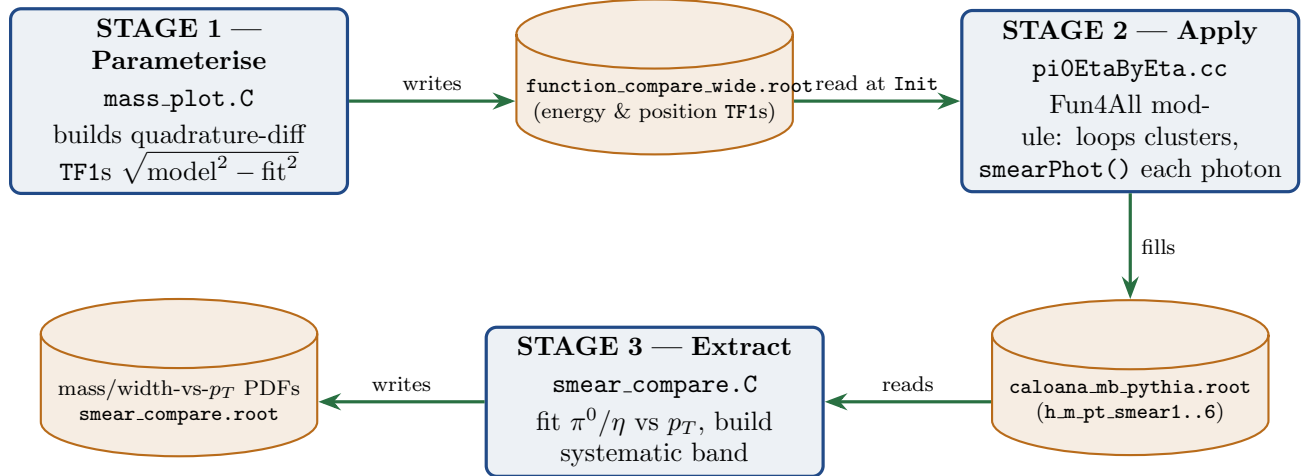


Figure 1: The production smearing chain. Stage 1 derives the per-energy smearing widths and writes them as TF1s. Stage 2 (the engine this document focuses on) reads that file at `Init` and applies the smear photon-by-photon during reconstruction. Stage 3 fits the smeared meson peaks and forms the smearing systematic band.

Stage 1 — mass_plot.C

From data-vs-MC resolution models and the fitted resolution, it builds the quadrature-difference TF1s of Eq. (2) for *both* energy and position, and writes them (names like `f_energy_difference_global_minimum`, `f_energy_difference_cE_0p08`, `f_energy_difference_cE_0p05`) into `function_compare_wide.root`.

Stage 2 — pi0EtaByEta.cc

The production smearing *engine* — a `Fun4All SubsysReco` module. It reads the TF1s once at `Init`, then in the event loop smears every photon pair and fills the `h_m_pt_smear` histograms. Detailed in Sec. 4.

Stage 3 — smear_compare.C

Reads the smeared histograms (`h_m_pt_smear12/13/14 A/ $\sqrt{E}$` variations), fits π^0 and η peaks vs p_T with a `MesonFitter`, and produces the mass-scale and width comparison plus the max–min systematic band.

4 The engine in detail: pi0EtaByEta.cc

This is a standard sPHENIX Fun4All reconstruction module with the usual life cycle: `Init()` (book histograms, load parameters) $\rightarrow$ `process_event()` (per-event work) $\rightarrow$ `End()` (write file). The smearing touches three of these places.

Init(): open `function_compare_wide.root`; `Clone()` the three energy TF1s into `f_energy_smear[0..2]` and three position TF1s into `f_position_smear[0..2]` (with legacy-name and bare-name fallbacks). Book `h_m_pt_smear[0..5]`.

process_event() $\rightarrow$ cluster pair loop: build `photon1`, `photon2` (`SetPtEtaPhiE`); form unsmeared `pi0 = photon1 + photon2`; apply p_T /asym cuts; fill the *nominal* mass histograms.

Smear sub-loop for `ks = 0..5`: `smearP1 = smearPhot(photon1, ks)`; `smearP2 = smearPhot(photon2, ks)`; `smearPi0 = smearP1 + smearP2`; fill `h_m_pt_smear[ks]`, `h_m_pt_eta_smear[ks]`, `h_dR_pi0_pt_smear`.

smearPhot(p, idx): the actual smearing kernel (Eq. (1)–(3)). Returns a new `TLorentzVector` with smeared E and (η, ϕ) .

End(): write the output ROOT file containing all `h_m_pt_smear*` histograms.

Figure 2: Control flow inside the production engine. The smear functions are loaded *once* at `Init` and `Eval`'d *per photon* inside the event loop.

4.1 Loading the smear curves (Init, around line 206)

The module hard-codes the parameter-file path and clones the TF1s into member arrays. The fallbacks let it survive renamed/older files.

```

206  const std::string kSmearParFile =
207      "/sphenix/u/bseidlitz/work/emcPerf/macros/figMaker/output/
        function_compare_wide.root";
208  TFile* smear_fin = TFile::Open(kSmearParFile.c_str(), "READ");
209  // for k = 0..2: get f_energy_difference_<model>, else legacy g_..., else bare
        name
210  //   f_energy_smear[k]   = (TF1*) fe->Clone(...);

```

```
211 // f_position_smear[k] = (TF1*) fp->Clone(...);
```

The three indices $k = 0, 1, 2$ correspond to the three resolution *models*: *global-minimum*, $c_E = 0.08$, and $c_E = 0.05$.

4.2 Booking the smeared histograms (Init, line 159)

```
159 h_m_pt_smear[i] = new TH2F(Form("h_m_pt_smear%d", i+1), "", 500,0,1.0,
    80,0,20);
160 h_m_pT_eta_smear[i] = new TH3F(Form("h_m_pT_eta_smear%d", i+1), "", 500,0,1.0,
    80,0,20, 96,0,96);
```

Six histograms ($\text{mass} \times p_T$, and $\text{mass} \times p_T \times i\eta$) — one per smear index. Mass axis 0–1 GeV, p_T axis 0–20 GeV.

4.3 Applying the smear in the event loop (line 738)

For every accepted cluster pair, all six smearing variations are run and filled:

```
738 for (int ks = 0; ks < 6; ++ks) {
739     TLorentzVector smearP1 = smearPhot(photon1, ks);
740     TLorentzVector smearP2 = smearPhot(photon2, ks);
741     TLorentzVector smearPi0 = smearP1 + smearP2;
742     h_m_pt_smear[ks]->Fill(smearPi0.M(), smearPi0.Pt(), eventWeight);
743     h_m_pT_eta_smear[ks]->Fill(smearPi0.M(), smearPi0.Pt(), (double)lt_eta,
        eventWeight);
744     h_dR_pi0_pt_smear->Fill(smearP1.DeltaR(smearP2), smearPi0.Pt(), eventWeight);
745 }
```

4.4 The smearing kernel: smearPhot() (line 863)

This is the heart of the system. The 6 indices decode as **3 models** $\times$ **2** (with / without an extra 0.08 constant term):

```
863 TLorentzVector pi0EtaByEta::smearPhot(TLorentzVector p, int smear_idx) {
864     if (smear_idx < 0 || smear_idx > 5) smear_idx = 0;
865     const int base_smear_idx = smear_idx % 3; // which of 3 models
866     const bool add_extra_const = (smear_idx >= 3); // 3..5 add an 0.08 const term
867
868     const float E = p.E();
869     if (E <= 0.f) return p;
870
871     float E_res = 0.08f; // fallbacks if TF1 missing/invalid
872     float P_res = 0.0001f;
873
874     if (f_energy_smear[base_smear_idx]) {
875         double s = f_energy_smear[base_smear_idx]->Eval(E); // sigma_extra(E)
876         if (std::isfinite(s) && s > 0.0) E_res = (float)s;
877     }
878     if (add_extra_const)
879         E_res = std::sqrt(0.08f*0.08f + E_res*E_res); // quadrature extra
        const
880
881     if (f_position_smear[base_smear_idx]) {
882         double s = f_position_smear[base_smear_idx]->Eval(E); // sigma_psi(E)
883         if (std::isfinite(s) && s > 0.0) P_res = (float)s;
```

```

884 }
885
886 float Efac      = rnd->Gaus(1.0f, E_res);    // MULTIPLICATIVE energy smear
887 float Pfac_phi  = rnd->Gaus(0.0f, P_res);    // ADDITIVE angle smear
888 float Pfac_eta  = rnd->Gaus(0.0f, P_res);
889 if (Efac < 0.f) Efac = 0.f;
890
891 float eta_s = p.Eta() + Pfac_eta;
892 float phi_s = p.Phi() + Pfac_phi;           // wrapped to (-pi, pi]
893 const float E_s = E * Efac;
894 TLorentzVector sp;
895 sp.SetPtEtaPhiE(E_s/std::cosh(eta_s), eta_s, phi_s, E_s);
896 return sp;
897 }

```

Reading this in plain English:

1. Pick the model ($\text{idx} \bmod 3$) and decide whether to add the extra 0.08 constant ($\text{idx} \geq 3$).
2. Evaluate the stored energy-smear TF1 at this photon's E to get $\sigma_{\text{extra}}(E)$ (Eq. (2)); optionally fold in $\sqrt{0.08^2 + \sigma^2}$.
3. Evaluate the position-smear TF1 to get the angular width $\sigma_\psi(E)$.
4. **Energy** is smeared *multiplicatively*: $E \rightarrow E \cdot \mathcal{N}(1, \sigma_{\text{extra}})$ (clamped ≥ 0).
5. **Direction** is smeared *additively*: $\eta \rightarrow \eta + \mathcal{N}(0, \sigma_\psi)$, $\phi \rightarrow \phi + \mathcal{N}(0, \sigma_\psi)$ (with ϕ wrapped).
6. Rebuild and return a `TLorentzVector` from the smeared (E_s, η_s, ϕ_s) .

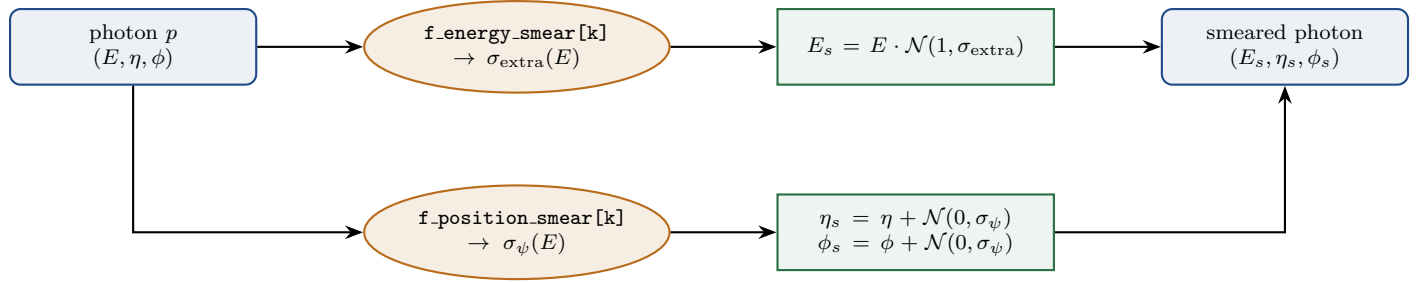


Figure 3: Inside `smearPhot()`: energy is multiplied by a Gaussian centred at 1 (width from the energy TF1); the two angles are shifted by Gaussians centred at 0 (width from the position TF1).

4.5 Decoding the six smear indices

smear_idx	base=idx%3	resolution model	extra 0.08 const?
0	0	global-minimum	no
1	1	$c_E = 0.08$	no
2	2	$c_E = 0.05$	no
3	0	global-minimum	yes ($\sqrt{0.08^2 + \sigma^2}$)
4	1	$c_E = 0.08$	yes
5	2	$c_E = 0.05$	yes

Table 2: The 6 smearing variations = 3 models $\times$ {with, without} an extra constant term. Running all 6 lets Stage 3 build a systematic band by taking the spread across variations.

5 Where CaloCalibEmc_Pi0 fits in

CaloCalibEmc_Pi0 (.cc/.h) is the *calibration* sibling module driven by the macro macro/Fun4All_G4_Pi0_Tbt.C. It is a SubsysReco that loops clusters, builds π^0 candidates, and fits the mass tower-by-tower / η -by- η to derive the EMCal energy-scale calibration constants (default target mass 0.152 GeV “to match sim”). It is the *energy-scale* half of the data/MC matching; pi0EtaByEta is the *resolution* half. Together they implement the two knobs the calibration talks turn: the scale $E_{\text{scale}} = f(E) \times 0.998$ and the resolution smearing of Eq. (1). The build is autotools-based (autogen.sh $\rightarrow$ configure $\rightarrow$ make install); jobs are launched through the Fun4All macro plus run_f4a.csh and Condor.

6 Standalone toy cross-checks (not the production result)

Two self-contained ROOT macros validate the concept independently:

eta_decay_simulation_with_smearing.C

A pure toy $\eta \rightarrow \gamma\gamma$ generator: throw η with a p_T^{-4} spectrum, decay isotropically, boost to the lab, smear each photon by $1 + \mathcal{N}(0, \sqrt{c^2 + a^2/E})$ with $a = 0.15$, $c = 0.12$, and re-form $M_{\gamma\gamma}$. Demonstrates how a chosen resolution maps onto the reconstructed mass peak and width.

resolutionsim.C

A toy *unfolding-bias* study: for several (MC-reso, data-reso) pairs it trains a RooUnfold response from a smeared spectrum, unfolds a “data” pass smeared differently, and plots the residual bias — quantifying how a data/MC resolution *mismatch* propagates through Bayesian unfolding.

7 How to run the chain (summary)

1. **Build once:** ./autogen.sh && ./configure --prefix=\$MYINSTALL && make install
2. **Stage 1:** root -l -b -q 'mass_plot.C' $\Rightarrow$ function_compare_wide.root
3. **Stage 2:** root -l -b -q 'Fun4All_G4_Pi0_Tbt.C(...)' (or run_f4a.csh+Condor) $\Rightarrow$ caloana_mb_pythia.root
4. **Stage 3:** root -l -b -q 'smear_compare.C' $\Rightarrow$ mass/width-vs- p_T PDFs

One-line dependency map

```
mass_plot.C  $\rightarrow$  function_compare_wide.root  $\rightarrow$  (Init) pi0EtaByEta::smearPhot()  $\rightarrow$ 
h_mpt_smear1..6  $\rightarrow$  smear_compare.C + MesonFitter  $\rightarrow$  mass/width vs  $p_T$ 
```

Key takeaways

- The smear “parameters” are the three terms (p_0, p_1, p_2) of Eq. (1); smearing widens MC by the *quadrature difference* to data (Eq. (2)).
- Energy smear is *multiplicative* ($\mathcal{N}(1, \sigma)$); position smear is *additive* ($\mathcal{N}(0, \sigma_\psi)$ on η and ϕ).
- The smearing widths live in a ROOT TF1 file produced offline; the engine just **Evals** them per photon. *The file is the interface between stages.*
- Six smear indices = 3 resolution models $\times$ {with/without an extra 0.08 constant}; the spread across them feeds the systematic band.
- pi0EtaByEta handles *resolution*; CaloCalibEmc_Pi0 handles *energy scale*; together they match MC to Run-24 data.